

Why NAMESPACE Cannot Replace renv

Understanding Version Management in R

PHB 243b: Practicum in Biostatistics

2026-01-13

Contents

1	The Question	1
2	Where Version Information Lives	1
3	Why == in DESCRIPTION Falls Short	2
3.1	Problem 1: Only Enforces Direct Dependencies	2
3.2	Problem 2: No Installation Mechanism	2
3.3	Problem 3: Conflicts in Shared Environments	2
3.4	Problem 4: R Version Not Captured	2
4	Comparison Summary	3
5	Conclusion	3

1 The Question

Can we use exact version specifications in DESCRIPTION instead of renv?

```
Imports:
  dplyr (== 1.1.4)
```

The short answer: **No**. This approach has fundamental limitations that make it unsuitable as a replacement for renv.

2 Where Version Information Lives

File	Purpose	Example
NAMESPACE	Symbol binding	importFrom(dplyr, filter)
DESCRIPTION	Minimum version requirements	Imports: dplyr (>= 1.1.0)
renv.lock	Exact version pinning	"dplyr": {"Version": "1.1.4"}

The NAMESPACE file cannot include version numbers at all—it only handles symbol-level declarations (what functions to import/export).

3 Why == in DESCRIPTION Falls Short

3.1 Problem 1: Only Enforces Direct Dependencies

```
Imports:
  dplyr (== 1.1.4)
```

This pins `dplyr`, but what about `dplyr`'s dependencies? The `dplyr` package imports `vctrs`, `pillar`, `cli`, `rlang`, and others. Their versions remain uncontrolled. A breaking change in `rlang` 1.2.0 vs 1.1.0 can still break your analysis.

`renv.lock` captures the **entire dependency tree**—potentially dozens or hundreds of packages with exact versions.

```
# See the full dependency tree
pak::pkg_deps_tree("dplyr")
```

3.2 Problem 2: No Installation Mechanism

`DESCRIPTION` only *declares* requirements; it does not *provide* old versions. When you run `install.packages("dplyr")`, R fetches the current CRAN version regardless of what `DESCRIPTION` says.

If `dplyr` 1.1.4 is no longer on CRAN (superseded by 1.2.0), installation fails.

`renv` can:

- Restore from its local cache
- Install from the CRAN archive
- Install from a specified repository snapshot (e.g., Posit Public Package Manager)

```
# renv handles old versions automatically
renv::restore()
```

3.3 Problem 3: Conflicts in Shared Environments

If two packages both specify exact versions:

```
Package A: dplyr (== 1.1.4)
Package B: dplyr (== 1.0.10)
```

Both cannot be satisfied simultaneously. Installation fails entirely.

`renv` creates **project-local libraries**, so each project can have its own `dplyr` version without conflict.

```
# renv isolates each project
.libPaths()
# [1] "/project/renv/library/R-4.4/aarch64-apple-darwin20"
# [2] "/Library/Frameworks/R.framework/.../library"
```

3.4 Problem 4: R Version Not Captured

`DESCRIPTION` cannot specify the R version itself. An analysis run on R 4.3.2 may behave differently on R 4.5.0 (recall the `stringsAsFactors` change in R 4.0.0).

`renv.lock` records the R version:

```
{
  "R": {
    "Version": "4.4.2",
    "Repositories": [
      {
```

```

    "Name": "CRAN",
    "URL": "https://cloud.r-project.org"
  }
]
}
}

```

Docker enforces the exact R version at runtime.

4 Comparison Summary

Capability	DESCRIPTION with ==	renv
Pin direct dependencies	Yes	Yes
Pin transitive dependencies	No	Yes
Install old versions	No	Yes
Project-local libraries	No	Yes
Record R version	No	Yes
Work across projects	Conflicts	Isolated

5 Conclusion

The == operator in DESCRIPTION is rarely used because it creates brittleness without providing actual reproducibility. It declares a requirement that the standard tooling cannot fulfill.

renv solves the problem that == only pretends to address:

- Full dependency tree capture
- Automated restoration of exact versions
- Project isolation
- Integration with repository snapshots and Docker

For reproducible research, use DESCRIPTION with >= for compatibility bounds and renv.lock for exact reproducibility.